

An In-depth and Black-box Characterization of the Effects of Clock Glitches on 8-bit MCUs

Josep Balasch, Benedikt Gierlichs, and Ingrid Verbauwhede
Katholieke Universiteit Leuven, COSIC and IBBT
Kasteelpark Arenberg 10, B-3001 Leuven-Heverlee, Belgium
Email: firstname.lastname@esat.kuleuven.be

Abstract—The literature about fault analysis typically describes fault injection mechanisms, e.g. glitches and lasers, and cryptanalytic techniques to exploit faults based on some assumed fault model. Our work narrows the gap between both topics. We thoroughly analyse *how* clock glitches affect a commercial low-cost processor by performing a large number of experiments on five devices. We observe that the effects of fault injection on two-stage pipeline devices are more complex than commonly reported in the literature. While injecting a fault is relatively easy, injecting an exploitable fault is hard. We further observe that the easiest to inject and reliable fault is to replace instructions, and that random faults do not occur. Finally, we explain how typical fault attacks can be mounted on this device, and we describe a new attack for which the fault injection is easy and the cryptanalysis trivial.

Keywords—Fault effect characterization; AVR MCU; clock glitches

I. INTRODUCTION

Physical attacks came to the attention of the scientific community in the late 1990's. Today it is a well known fact that variations of e.g. the execution time [1], the instantaneous power consumption [2] and the electromagnetic emanations [3], [4] of a device implementing a cryptographic algorithm leak information about secret data. Attacks exploiting such information are called (passive) side channel attacks. It is further well understood that unusual conditions of the close, physical environment of a cryptographic implementation have an impact on the operation of a device. The latter can be exploited by an adversary that modifies the operating environment of a device temporarily to induce computational errors, known as transient faults. Such and other active attacks are fault attacks. Typical fault injection mechanisms include glitches on the clock signal or the power supply [5], and the photoelectric effects caused by lasers or white light [6]. The former mechanisms are typically inexpensive to implement but affect the entire chip, while laser setups are more expensive but allow to stimulate specific regions of a chip.

The first fault attack, published at Eurocrypt'97, is the Bellcore attack [7], targeting implementations of RSA-CRT. The attack is particularly powerful as any single fault injected into either of the CRT-branches allows to factorize the RSA modulus. Fault attacks quickly gained

importance due to their severe power to break cryptographic implementations.

Many later works focus on attacking implementations of symmetric-key algorithms. Today, we know about e.g. Differential Fault Analysis (DFA) [8], Collision Fault Analysis (CFA) [9] and Ineffective Fault Analysis (IFA) [10], [11]. These generic attack techniques all represent a union of physical fault injection and cryptanalytic fault exploitation. As a consequence, most publications in this area of research focus on either fault exploitation assuming that a certain fault pattern can be injected physically (fault model), or on how to physically inject faults, sometimes in combination with verifying a theoretical attack. At the same time, there is a rich body of literature about countermeasures against fault attacks [12]. Many of them are based on some form of redundant computation.

Related Work. Our work focuses on the case of non-invasive fault attacks, particularly, on the injection of faults via glitches in the external clock signal provided to a MicroController Unit (MCU). This type of attack has been successfully implemented using cryptographic hardware co-processors as target platforms [13], [14], [15]. However the effects of glitches (either in the clock signal or in the power supply) on software implementations in embedded MCUs have only been addressed in a handful of articles. Kömmerling and Kuhn [16] enumerate a series of potential vulnerabilities that can be triggered by inducing glitches. For instance, glitches can be used to extend the runtime of loops in serial port output routines to see more of the memory after the output buffer [17]; or they can also be used to reduce the number of loop iterations, e.g. to convert a secure iterated block cipher into weaker single-round variant [18]. Bar-El et al. [12] enumerate some effects observed on MCUs when inducing spikes on the power supply. Depending on some conditions, the authors are able to inject faults such that instructions are skipped or the data manipulated by the processor is modified. However, the technical details on how to perform such attacks, including the characteristics of the target MCU, are not provided in these papers.

Further work has focused on the injection of spikes in the power supply with the aim of skipping critical instructions:

Choukri and Tunstall [19] reduce the number of rounds of an AES implementation on a PIC16F877 smart card; Kim and Quisquater [20] skip two subroutine calls in an RSA-CRT implementation on an AVR MCU; similarly, Schmidt and Herbst [21] prevent a subroutine call in a square-and-multiply RSA implementation on an AVR MCU. Perhaps closer to our work, Barengi et. al [22] perform a study of the fault effects on a 32-bit multi-stage pipeline ARM MCU; the results are later used to break implementations of AES and RSA.

The goal of our work is to thoroughly analyse the effects of clock glitches on the instructions being executed by a MCU. To this end, we choose a legacy 8-bit AVR controller as target platform, a typical representative of low-cost embedded devices. This MCU allows a potential attacker to manipulate the external clock signal at will. It should be noted that secure smart cards have built-in countermeasures, typically an internal clock or sensors, in order to prevent fault injection via clock glitches. However, our research does not aim to propose or experimentally prove theoretical fault attacks on cryptographic implementations, but rather to answer questions such as how, when, and why do faults occur.

The two main contributions of this paper are, first, to provide a complete study and characterization of the effects of clock glitches on legacy smart card architectures without hardware coprocessor modules. The analysis is done in a black-box setting, and using as target platform an 8-bit AVR controller operating on a two-stage pipeline. And second, to put forward a more concrete foundation for future work in fault attacks and countermeasures, by highlighting which type of faulty behaviour can be expected and/or exploited as a result of glitches in the clock signal.

Outline. The rest of this paper is organized as follows. We describe our experimental setup and our target platform in Section II and Section III, respectively. The testing framework is introduced in Section IV. Section V describes the effects of fault injections on the program flow of an application, whereas Section VI focuses on the effects on the application's data flow. We summarize our findings and enumerate potential applications in Section VII, and conclude in Section VIII.

II. EXPERIMENTAL SETUP

The experimental setup used in this work is depicted in Figure 1. We have implemented a custom ISO/IEC 7816-3 [23] compliant smart card reader with a fully controllable clock signal using a Virtex-II Pro XC2VP30 FPGA [24]. The interface with the smart card at the link and physical layers follows the ISO/IEC 7816-3 standard, while the communication is performed via Application Protocol Data Units (APDUs) as specified in ISO/IEC 7816-4 [25]. The computer, acting as a user interface, communicates with the FPGA via an RS232 interface.

The FPGA behaves as an off-the-shelf smart card reader. The clock signal provided to the smart card has a fixed nominal frequency in accordance to the MCU specifications. The APDU commands exchanged between the computer and the smart card during communication are simply forwarded by the FPGA to the end receiver. Note that the T0 and T1 protocols specified in ISO/IEC 7816-3 are based on request/response commands, i.e. the computer is always the device that triggers an action of the smart card.

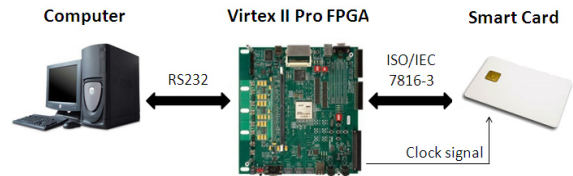


Figure 1. Experimental setup.

Glitch generation. The effect of injecting a glitch in the clock signal is depicted in Figure 2. We denote the nominal period of the clock signal as T_n and the period (or duration) of a glitch as T_g . The idea of injecting a glitch is to temporarily overclock the smart card, i.e. to insert a clock period such that $T_g \ll T_n$ that potentially causes a transient malfunction of the MCU. Notice that after injecting a glitch, the following clock period is reduced from T_n to $T_n - T_g$ in our setup. However, given that $T_g \ll T_n$, this “post-glitch” period does not affect the normal behaviour of the MCU.

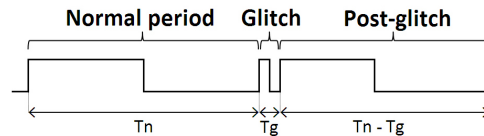


Figure 2. Injection of a glitch in the clock signal.

We have developed two different mechanisms to introduce glitches in the clock signal. In the first mechanism, illustrated in Figure 3, the FPGA generates the *output CLK* fed to the smart card using a combination of two reference signals denoted as *nominal CLK* (with period T_n) and *high-freq. CLK* (with period T_g). Glitches in the frequency of *output CLK* are injected when indicated by the *selection* signal. For the sake of reproducibility, both *nominal CLK* and *high-freq. CLK* signals have to be perfectly phase-aligned. This can be easily achieved by generating the *nominal CLK* signal from the *high-freq. CLK* signal, which in turn determines the granularity of the glitch width. For instance, by fixing the frequency of the *nominal CLK* to 1 MHz (such that T_n is $1\mu s$), the possible frequency values of the *high-freq. CLK* are tied to be multiples of 2 MHz. In other words, the set of possible glitch periods is given by T_g (in μs) = $1/2^i$,

where $i = 1, 2, 3, \dots$. The accuracy of the glitch period has a standard deviation of 60 ps.

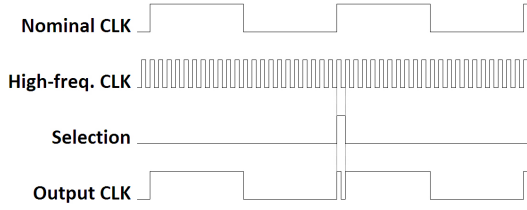


Figure 3. Glitch generation using high-frequency signal.

In the second mechanism, shown in Figure 4, we use a similar approach as in [15], [26]. In this case, a glitch in *output CLK* is generated by switching between three signals with the same period T_n but with different phases. The advantage of this mechanism w.r.t. the first one is that it provides more granularity in the glitch period, in particular for low frequencies. We can increase the glitch period in steps of approximately 1 ns such that the set of possible glitch periods is given by T_g (in ns) = i , where $i = 1, 2, 3, \dots$. The standard deviation of the glitch period for this mechanism is 70 ps.

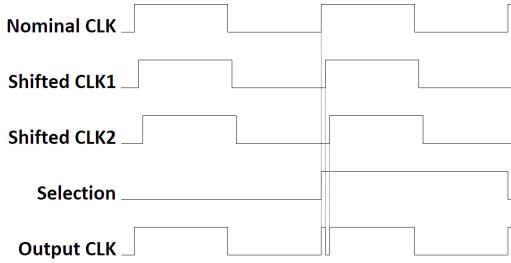


Figure 4. Glitch generation using phase-shifted signals.

As a final comment, note that the *selection* signal in these mechanisms allows a wide range of glitch injection patterns, which are by no means restricted to one glitch per trial execution. The selection of all parameters involved in the generation of the output clock signal (e.g. glitch mechanism, nominal period T_n , glitch period T_g , glitch position, etc.), is completely configurable by the user via commands send from the PC to the FPGA, thus obtaining a highly-flexible yet automatized experimental setup to carry out our study.

III. TARGET PLATFORM

We choose as target platform a MCU belonging to the 8-bit Atmel AVR family, namely the ATmega163 microcontroller [27]. There are several reasons for this choice. First, and most important, this device operates on an external clock signal, such that it is possible to inject faults to the device using this interface. Second, the characterization of the effects of fault injection on AVR MCUs is a challenging

task: AVR controllers have a modified Harvard architecture, i.e. although access to program code (flash memory) and data (internal SRAM) is physically separated in the chip (strict Harvard architecture), the CPU can concurrently use both buses in a clock cycle. This characteristic, combined with a RISC architecture with most of the instructions executing in a single-cycle, allows to obtain a two-stage pipeline: while one instruction is being executed, the next one is pre-fetched from program memory as shown in Figure 5. In contrast to von Neumann architectures (e.g. 8051 MCUs), several critical operations happen in parallel during one clock cycle. Consequently, fault injections can have multiple and complex effects.

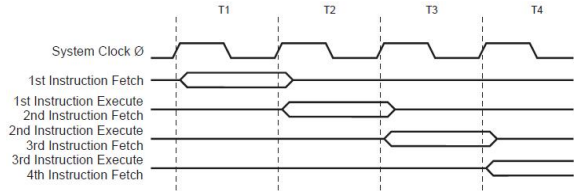


Figure 5. Pipeline in AVR controllers (source: ATmega163 datasheet).

Finally, Atmel AVR controllers are known devices largely used in the related literature, not only in fault analysis [20], [21], but also in side channel attacks [28]. So far no study has been made to fully characterize and understand the reaction of these devices to fault injection via clock glitches. We aim to fill this gap in the following sections.

We stress that although Atmel offers a family of AVR MCUs specifically designed for security applications [29], the smart cards used in our tests have no security claims whatsoever. Our research motivation is not to evaluate the level of resistance of such MCUs to fault attacks, but rather to understand and characterize the effects of fault injection via clock glitches on one model of the low-cost family. Note that the analysis is done in a black-box setting, i.e. we only have access to the publicly available data sheets.

IV. TESTING FRAMEWORK

The approach followed in our experiments consists in decreasing the glitch period T_g , starting with a value such as 125 ns (or 8 MHz) for which the MCU functions correctly, until 15 ns. This lower bound is determined by the switching speed of the FPGA board's I/O pins as well as some external analogue circuitry of our experimental setup. When faults start occurring, we analyse them in order to be able to characterize the chip's behaviour. Our experiments show that the critical path (i.e. maximum frequency tolerated by the MCU) is determined by the access to Program Memory. In other words, the first effect noticed when decreasing the glitch period is an erroneous behaviour of the pre-fetching stage.

In the following we will make a distinction between which pipeline stage is affected by the glitch. We will begin by

focusing on the effects of clock glitches on the pre-fetching stage, analysing how it is possible to inject faults such that the *program flow* is altered. After this, we will focus on the effects of the glitches on the execution stage, studying how the expected *data flow* of a program is changed. In our experiments we have used a total of five ATMega163 smart cards to verify that they all respond to fault injection in a very similar way.

We have implemented several test applications¹ in assembly language and executed them a large number of times in order to obtain and analyse the effects of clock glitches. In order to make the interpretation of the results more clear, we provide some exemplary code fragments. Although such tests do not correspond to any particular cryptographic implementation, the results obtained can be easily and perfectly extrapolated to the general case.

Finally, note that by targeting a device with a two-stage pipeline and without access to details of the inner workings of the MCU, the analysis of the faults' outcome becomes an arduous task. The only information available for the interpretation of the faults's effects consists of an array of output data. Before running a test, we bring the MCU to a state $\mathcal{A}$ such that all possible variables (SRAM values, program memory, registers, flags, and even room temperature) are fixed and known. A normal execution of the test application brings the MCU to an "expected" state $\mathcal{B}$, whereas a faulty execution brings it to an "incorrect" state $\mathcal{B}'$. Manually reverse-engineering the chain of events that explains the transition from state $\mathcal{B}$ to state $\mathcal{B}'$ is far from trivial.

V. EFFECTS OF CLOCK GLITCHES ON PROGRAM FLOW

The AVR instruction set consists of 130 commands, most of them executing within a single clock cycle. Instruction opcodes are typically encoded and stored in Program Memory in 16-bit words. Although being an 8-bit device, the AVR MCU has a 16-bit Program Bus. This means that in the pre-fetching phase, the 16-bit opcode pointed to by the Program Counter (PC) is loaded at once. In turn, the PC is also incremented in this stage, such that the next opcode is correctly loaded in the following clock cycle. The behaviour of multi-cycle instructions differs from that of single-cycle instructions; these differences will be discussed later.

A. NOP: No Operation

We start our analysis by testing the effects on the most simple command available, namely NOP. As this instruction does not perform any operation in the execution phase, glitches will only affect the pre-fetching stage. Our first test is depicted in Figure 6, where *Inst* refers to **any** of the available AVR instructions. By injecting a glitch in clock

cycle i (when NOP is being executed), one can possibly cause an erroneous behaviour in the pre-fetching phase.

Cycle	Instruction	Opcode (bin)
i	NOP	0000 0000 0000 0000
$i+1$	<i>Inst</i>	-

Figure 6. Code example for NOP (I).

For testing purposes, let us assume that *Inst* is the command EOR R15,R5 (Exclusive OR) as illustrated in the top part of Figure 7. When injecting a glitch with period smaller than or equal to 59 ns in clock cycle i , we observe that EOR R15,R5 is never executed. Intuitively, one can assume that the MCU does not have time to load the next command from Program Memory as a consequence of the glitch. So a reasonable explanation is that the opcode being executed at the time of the glitch (e.g. NOP in cycle i) is executed again in cycle $i+1$, as shown in the lower part of Figure 7. Note however, that the PC is clearly not affected by the glitch and is correctly incremented. Otherwise, the MCU would simply pre-fetch the command EOR R15,R5 in cycle $i+1$ and execute it in cycle $i+2$. A likely explanation for the PC not being affected by the glitch in any of our experiments is that incrementing the PC simply requires less time than 15 ns.

Glitch period	Cycle	Instruction	Opcode (bin)
-	i	NOP	0000 0000 0000 0000
-	$i+1$	EOR R15,R5	0010 0100 1111 0101
≤ 59 ns	$i+1$	NOP	0000 0000 0000 0000

Figure 7. Code example for NOP (II).

Suppose now that *Inst* is the command SER R18 (Set Bits in Register) as illustrated in Figure 8. In this case, we observe that depending on the glitch period the command SER R18 is substituted by instructions other than NOP. In particular, for a glitch period equal to 61 ns the command LDI R18,0xEF (Load Immediate to Register) is executed. Decreasing the glitch period to 60 ns produces the appearance of the command SBC R12,R15 (Subtract with Carry). Finally, for any glitch period smaller than or equal to 59 ns, we observe the same effect as shown in Figure 7, namely, NOP is executed.

Glitch period	Cycle	Instruction	Opcode (bin)
-	i	NOP	0000 0000 0000 0000
-	$i+1$	SER R18	1110 1111 0010 1111
≤ 61 ns	$i+1$	LDI R18,0xEF	1110 1110 0010 1111
≤ 60 ns	$i+1$	SBC R12,R15	0000 1000 0010 1111
≤ 59 ns	$i+1$	NOP	0000 0000 0000 0000

Figure 8. Code example for NOP (III).

These results shown in Figure 8 illustrate the transition in which the MCU internally updates the opcode to be executed. As one can notice, there is a progression from the

¹These routines have been executed using a modified version of the T0 protocol compliant Simple Operating System for Smartcard Education (SOSSE), available here: <http://www.mbsks.franken.de/sosse/>

expected command (SER R18) to the previous command (NOP), in the sense that more bits of the erroneous opcodes are degraded to zero as the glitch period decreases. Strictly speaking, at this point it is not fully correct to describe the effect of the glitch as *skipping* an instruction; rather differently, as another command is executed instead of the expected instruction, a more accurate description of the glitch effect would be *replacing* an instruction.

Note that when LDI or SBC are executed instead of SER, some registers are overwritten with the values resulting from the execution of such commands; thus, a single fault injection disrupts at the same time both the program flow and the data flow.

The effects depicted in Figure 8 are observed in all five ATMega163 smart cards, although there are some slight differences. First, the glitch periods for which instructions are replaced can vary from card to card. And second, it is possible that instructions different than those in Figure 8 appear in cycle $i+1$. We have however verified that for glitch widths smaller than approximately 52 ns a NOP is always effectively executed in all cards.

A particularly interesting case in the test is observed when the command Inst has a 32-bit opcode. Consider, as shown in Figure 9, that this instruction is LDS R22,0x0128 (Load Direct From Data Space). As the program bus of the AVR is 16-bit wide, LDS requires an extra cycle to fetch the second half of the opcode, i.e. the value 0x0128, from Program Memory. By injecting a clock glitch with period 59 ns in cycle i , LDS is replaced by NOP in cycle $i+1$. However, as the skipped command has a 32-bit opcode, the value 0x0128 is pre-fetched from Program Memory in cycle $i+1$ and interpreted as a command in cycle $i+2$. As a result, a completely wrong instruction is inserted in the program flow. For this particular example, the instruction corresponding to opcode 0x0128 is MOVW R4,R16 (Copy Register Word), which moves a 16-bit word from a pair of registers to another pair of registers in a single cycle.

Glitch period	Cycle	Instruction	Opcode (bin)
-	i	NOP	0000 0000 0000 0000
	$i+1$	LDS R22,0x0128	1001 0001 0110 0000
	$i+2$		0000 0001 0010 1000
≤ 59 ns	$i+1$	NOP	0000 0000 0000 0000
	$i+2$	MOVW R4,R16	0000 0001 0010 1000

Figure 9. Code example for NOP (IV).

If the second half of the 32-bit opcode is not a valid command, for instance, LDS R22,0x0060, the MCU will execute the opcode 0x0060 in cycle $i+2$ as a consequence of the fault. However, the execution of illegal opcodes in AVR MCUs is carried out without affecting the program flow; in fact, they have the same effect as NOPs².

²It is possible that other MCUs behave differently when interpreting an invalid opcode, for example, resetting the chip. For the AVR controllers we have used this is however not the case observed.

B. Branching instructions

For the second set of experiments we target branching instructions. These commands do not have an execution phase that directly affects data; instead, they modify the value of the PC according to a tested condition. Consider the code example shown in the top part of Figure 10. In cycle i , the TST command checks whether register R12 holds a value equal to zero. If so, it sets the Zero flag in the Status Register (SREG); otherwise, the flag is cleared. In cycle $i+1$, the BREQ (Branch if Equal) command checks the value of the Zero flag: if the flag is set, it modifies the value of the PC in order to branch to a different code segment; otherwise, PC is incremented such that the next instruction is SER R26. The former option requires two cycles to complete, while the latter executes in a single cycle.

If the Zero flag is cleared, BREQ simply increments the PC in a single cycle, thus behaving similarly to a NOP. By injecting a fault in cycle $i+1$ one would expect a faulty behaviour such as in the previous experiments. However, as shown in Figure 10, the amount of faulty instructions executed instead of SER R26 and the glitch periods for which errors appear are quite different.

Glitch period	Cycle	Instruction	Opcode (bin)
-	i	TST R12	0010 0000 1100 1100
	$i+1$	BREQ PC+0x02	1111 0000 0000 1001
	$i+2$	SER R26	1110 1111 1010 1111
≤ 57 ns	$i+2$	LDI R26,0xEF	1110 1110 1010 1111
≤ 56 ns	$i+2$	LDI R26,0xCF	1110 1100 1010 1111
≤ 52 ns	$i+2$	LDI R26,0x0F	1110 0000 1010 1111
≤ 45 ns	$i+2$	LDI R16,0x09	1110 0000 0000 1001
≤ 32 ns	$i+2$	LD R0,Y+0x01	1000 0000 0000 1001
≤ 28 ns	$i+2$	LD R0,Y	1000 0000 0000 1000
≤ 27 ns	$i+2$	LDI R16,0x09	1110 0000 0000 1001
≤ 15 ns	$i+2$	BREQ PC+0x02	1111 0000 0000 1001

Figure 10. Code example for BREQ.

In the range $57 \text{ ns} \leq T_g \leq 28 \text{ ns}$ the opcode values describe a clear transition towards zero, i.e. stuck-at-zero pattern. The first erroneous command to be executed (LDI R26,0xEF) differs from the expected (SER R26) in that bit 8 is zero instead of one; for the second command (LDI R26,0xCF) bit 9 is also cleared. This progression is observed until for LD R0,Y a total of 11 bits are cleared compared to those of SER R26.

By decreasing the glitch period further than 27 ns, one would expect to obtain an erroneous opcode consisting of only zeroes. However, results show that after LD R0,Y (with only two bits set to one), there is a transition to LDI R16,0x09 (with five bits set to one). Finally, for a glitch period equal to 15 ns, the command BREQ PC+0x02 is executed again instead of SER R26.

At this point, it is clear that the figure shows the transition from the expected opcode to the previous opcode, most probably going through an intermediate all-zero state. Depending on the period of the glitch injected, the instruction

executed in cycle $i+2$ is a degraded version of either the expected opcode (SER R26) or the previous opcode (BREQ PC+0x02). Once again, although the general behaviour observed in other cards is the same, the faulty instructions and the glitch periods for which they are observed might vary with respect to the ones given in Figure 10.

C. Single-cycle instructions

In order to prevent effects on the data flow, skipped instructions should ideally be replaced by either NOPs, illegal commands, or even testing commands (such as TST). However, the amount of variables that determine the final shape of the faulty opcode, as well as the differences observed for different ATMega163 cards, make it very difficult to characterize and determine the outcome of the fault.

One way to obtain a rough estimation of the probability that faulty commands affecting the data flow appear, consists in running a series of tests similar to the one depicted in Figure 11. This code fragment includes only single-cycle instructions, such that a pre-fetching stage is performed in each clock cycle. By injecting a fault in each cycle of these test programs, one can count how many times the data flow is affected, i.e. how many times the faulty instruction affects internal data variables. We have generated several of these tests, changing parameters such as the order of the instructions and the source/destination registers in order to maximize the variation of the opcodes. The results obtained indicate that around 50% of the time, the injection of a clock glitch with period $57 \text{ ns} \leq T_g \leq 28 \text{ ns}$ results in the insertion of a faulty command whose execution alters the data flow.

Glitch period	Cycle	Instruction	Opcode (bin)
	i	SUB R7, R5	0001 1000 0111 0101
	$i+1$	ADD R8, R4	0000 1100 1000 0100
-	$i+2$	LSL R15	0000 1100 1111 1111
		...	
	$i+20$	MOV R27, R6	0010 1101 1011 0110

Figure 11. Code example for single-cycle instructions.

D. Multi-cycle instructions

As mentioned earlier, multi-cycle instructions have a different behaviour than single-cycle commands in the sense that they do not follow exactly the two-stage pipeline as shown in Figure 5. Some of these instructions perform the pre-fetching stage in the last execution cycle. In such cases, it is possible to inject faults with a similar effect as observed earlier, i.e. the following instruction is replaced by another instruction. This is the case for commands such as RET (Return from Subroutine) or branching instructions such as BREQ (Branch if Equal).

We have however noticed that not all multi-cycle instructions behave like this. For instance, we have not been able to replace instructions executed after commands such as LD

(Load From Data Space), LPM (Load from Program Memory) or RCALL (Relative Call to Subroutine), independently of the glitch width and of the cycle in which the fault is injected.

VI. EFFECTS OF CLOCK GLITCHES ON DATA FLOW

Until now we have exclusively focused on how clock glitches affect the program flow of certain commands. However, and given the two-stage pipeline of our target MCU, it might be possible to affect also the execution stage of certain instructions such that the resulting computation is corrupted. In this section we address the feasibility and the conditions under which these faults occur.

A. Single-cycle instructions

We have observed that it is possible to affect the execution stage of single-cycle instructions operating on data (e.g. arithmetic, logic, and bit operations). However, there are some issues that make it difficult to characterize the effect of such fault injections. First, the glitch period for which corrupted values start appearing depends on the specific instruction being executed; second, the source and destination registers used as operands also influence the outcome of the fault; and third, the fact that a single fault affects both pipeline stages at the same time makes the interpretation of the fault more complex.

In general, a minimum glitch period of approximately 27 ns is required to affect the execution stage of most instructions. The first effect observed by decreasing the glitch period is that one or more bits of the correct result are stuck-at-zero. For smaller glitches, the result of the operation becomes an erroneous value independent of the correct result or the previous value in the register. This behaviour should however not be taken as a generalization. As mentioned, there are too many parameters that influence the exact outcome of the fault.

An important observation is that the value of the corrupt result does not vary for an arbitrary number of executions using the exact same MCU and glitch parameters. In other words, given a state $\mathcal{A}$ and a fixed glitch sequence, the effect of the fault is constant. This observation implies that the result of the fault injection should not be referred to as *random*, but rather *deterministic* for a series of fixed executions.

B. Multi-cycle instructions

Given the complexity of the results obtained for single-cycle instructions, one might wonder whether it is easier to target the execution stage of multi-cycle instructions. In this section we focus on two of the more relevant such instructions, namely, LD and LPM.

LD: Load from Data Space

The LD (Load from Data Space) instruction requires two clock cycles to execute. It loads the value pointed to by either

register X, Y, or Z into any destination register in the MCU. In order to better understand the internal behaviour of this instruction, we have collected and analysed different groups of power measurements while executing LD in the AVR controller. The result is illustrated in Figure 12, in which the execution of LD corresponds to time samples 80 to 180. The dashed curve indicates the difference of means of two sets of measurements for which a fixed value is loaded from two different memory positions. A peak is noticeable at the rising edge of the second cycle, thus verifying that the SRAM address is updated at this time. The solid curve indicates the difference of means of two sets of measurements loading different values from a fixed SRAM address. A peak is visible at the falling edge of the second cycle, indicating that the SRAM value is loaded in the data bus at this time.

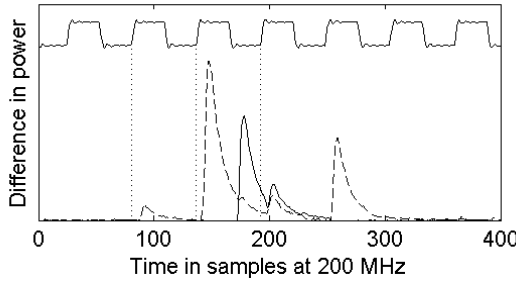


Figure 12. Difference of means for different executions of LD.

In order to evaluate the effects of clock glitches on the LD instruction, we run the test in Figure 13 for a fixed value in the pointer Z. We ensure that potential effects of the destination register on the outcome of the glitch are also taken into account by considering all possible cases.

Glitch period	Cycle	Instruction	Opcode (bin)
-	i	LD R0, Z	1000 0000 0000 0000
	$i+2$	LD R1, Z	1000 0000 0001 0000
	...	...	...
	$i+50$	LD R25, Z	1000 0111 1001 0000

Figure 13. Code example for LD.

For the first cycle, faulty values start to appear for glitch periods smaller than 24 ns. A clock glitch in this position produces an error in the value of the SRAM pointer; as a result, a value stored in a different SRAM address is loaded. We have verified this assumption by filling the SRAM space with known values. By repeating the same test multiple times we verify that a large percentage of the erroneous values come from this pre-filled SRAM space. We have not been able however to characterize how the pointer is affected by the glitch, i.e. the exact relation between the correct SRAM address, the faulty SRAM address, and the glitch period.

For the second clock cycle we begin to observe erroneous results when the glitch period is approximately 64 ns. In

the low period of this cycle, the SRAM value is transferred via the data bus. In AVR platforms the data bus is not pre-charged, meaning that the last value that uses the data bus is kept until overwritten. A glitch injection in the second cycle of LD prevents the value in the data bus from being updated. In particular, depending on the glitch period it is possible to prevent one or more bit transitions from happening.

Figure 14 shows the number of bit transitions that are avoided in function of the glitch width. The first effect produced by the fault is to prevent the transition of a single bit (bit 4 in our experiments) from 0 to 1. Consider that the previous value on the data bus is 0x00; if we try to load the value 0xFF while injecting a glitch, the outcome is the value 0xEF being loaded. Note that for the opposite case (previous value in the bus being 0xFF and trying to load 0x00), the glitch does not affect the result. This is because bit transitions are in this case from 1 to 0. As we decrease the width of the glitch more erroneous 0 → 1 transitions appear, until at around 55 ns no such transition is possible anymore.

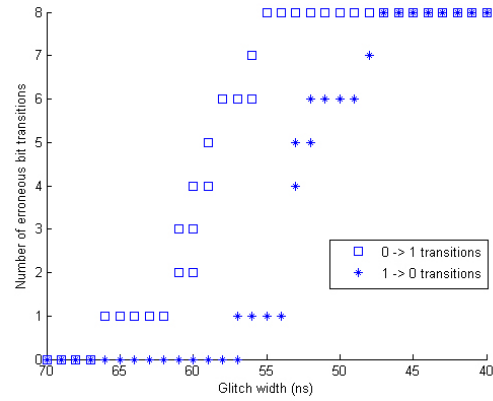


Figure 14. Number of erroneous transitions in the data bus when decreasing the glitch width.

Focusing on the other bit transition, namely 1 → 0, we observe that the faulty behaviour starts at around 54 ns. The first bit affected in our experiments is once again bit 4. Similarly to the 0 → 1 transition, more erroneous bit transitions appear by decreasing the glitch period. Finally, when T_g is around 46 ns, no bit transitions are possible at all. At this point the effect of the glitch is that the erroneous value being effectively loaded corresponds to the previous value that was on the bus. In other words, if we inject a glitch with period equal to or smaller than 46 ns in cycle $i+3$ in Fig. 13, the value that ends up in register R1 is the same as was previously loaded into R0.

Once again, although the transition in the data bus is clearly visible in all five ATMega163 cards tested, the results for a particular glitch period might vary. However, we have verified that for glitch widths smaller than or equal to 44 ns no bit transitions in the data bus are possible in any card;

also important, the first transition affected corresponds to bit 4 in all cards.

LPM: Load from Program Memory

The LPM instruction (Load from Program Memory) requires three clock cycles to execute. It loads the value in Program Memory pointed to by register Z into any of the registers in the MCU. In order to evaluate the effect of the clock glitch on the LPM instruction, we run the test in Figure 15 such that the effects on each destination register are also taken into account.

Glitch period	Cycle	Instruction	Opcode (bin)
	i	LPM R0, Z	1001 0000 0000 0100
	$i+3$	LPM R1, Z	1001 0000 0001 0100
	...	...	...
	$i+78$	LPM R25, Z	1001 0001 1001 0100

Figure 15. Code example for LPM.

By injecting a glitch with period smaller than 25 ns in the first cycle of LPM we observe that an erroneous value is loaded into the destination register. In this cycle, the LPM instruction sets the address to the program memory from which the value has to be loaded. The glitch alters the value of the address in a way that an erroneous code memory location is pointed to, thus resulting in a wrong value being stored into the register. We tested this hypothesis by filling the free space in Program Memory with known values (e.g. 0x4444, 0x5555, etc.). By repeating the tests multiple times, we observe that a large percentage of the values loaded corresponds to these pre-set constants, thus verifying our hypothesis.

In the second execution cycle of LPM, erroneous values start appearing for glitches smaller than or equal to 47 ns. We have observed that these faulty values correspond to one of the bytes of the next opcode in Program Memory. For instance, consider that we inject a glitch in cycle $i+2$ in the code example shown in Figure 15. The following opcode corresponds to the instruction LPM R1, Z, and its value is 0x9014. If the least significant bit of pointer Z is set to one when executing LPM R0, Z, then the erroneous value that ends up loaded in R0 is the upper byte of the next opcode, i.e. 0x90. If the least significant bit of the pointer Z is cleared, then the value that is loaded corresponds to 0x14.

This behaviour can be explained by the fact that, although the Program Bus is 16-bit wide, the value loaded by LPM is only 8-bits. Thus the least significant bit of the pointer Z is used to determine whether the upper byte or the lower byte is to be loaded into the destination register. By injecting a fault in the second cycle of LPM, the address to Program Memory is not updated with the value of pointer Z, such that it remains pointing at the following opcode to be loaded. That is the reason why the outcome of the fault consists in loading this particular erroneous value.

VII. SUMMARY AND APPLICATIONS OF THE RESULTS

Faults in program flow. We have observed that we can affect the fetching of the next opcode such that it is replaced by another instruction. Accurate timing would ideally allow to fetch an all-zero opcode (NOP), i.e. to effectively “skip” a command. However, controlling the opcode transition turns out to be highly complex. So typically, a faulty version of the old or the new opcode with some 1 bits set to 0 will be fetched depending on the glitch timing. In this case it is difficult to control the fault’s effect, since the faulty opcode can represent one of many valid instructions that manipulate data or program flow. Moreover, the transition steps (faulty opcodes executed) vary depending on the smart card used and the specific MCU state. Using very short glitch periods, we can potentially prevent the fetching of a new opcode entirely such that the old opcode is executed twice. However, when using such short glitch periods the fault also affects the execution stage of the current instruction. This is in particular problematic if the current opcode is a single-cycle command, as the fault affects both program flow and data flow at the same time. This leads to a more complex fault, possibly affecting data in several registers resp. at several memory addresses. We have estimated that around 50% of the times, a faulty command that does not affect the data flow appears as a replacement for the skipped instruction. So variations on the glitch period might allow to inject faults with potential for exploitation. However, we find it easier to replace instructions without disturbing the data flow for some multi-cycle instructions such as branching or return commands, i.e. when no data is manipulated and the pre-fetching of the new opcode happens in the last execution cycle.

These observed faults in the program flow should allow the typical applications: to skip checks and to prevent counters from increasing or decreasing, e.g. round counters or counters in I/O routines. In addition, new (somewhat trivial yet powerful) attacks can be mounted to attack cryptographic implementations. For illustrative purposes only, we will provide an example using AES-128 [30].

Suppose the four main steps of the cipher (ARK, SB, SR and MC), and possibly the key schedule, are implemented as subroutines that the main function simply calls as required using RCALL. Consider that the SR subroutine has been called in the last round of encryption and is executing the final RET instruction to return to the main function. We can easily inject a fault into the last cycle of the four-cycle RET instruction that affects the fetching of the new opcode (it would be the next RCALL instruction, here calling ARK for the last time) and replaces it with an invalid opcode or with NOP, effectively skipping the subroutine call. An XOR of the obtained faulty ciphertext and the correct ciphertext yields the full last round key. Note that even if the implementation is organized differently, i.e. unrolled

instead of using `RCALLs`, the fact that our setup supports the injection of faults into any arbitrary sequence of cycles still allows to perform the attack. For instance, we can then inject a series of consecutive faults into *all* cycles following the last SR operation to effectively skip each and every instruction performing the last ARK. Varying the glitch period might help the attacker in distinguishing whether faulty opcodes changing the data flow have been inserted. We have fully implemented and tested this trivial attack in all five smart card in several repetitions, thus confirming its feasibility.

Faults in data flow. We have further observed that it is possible to inject faults into the execution of instructions. For single-cycle instructions, the required glitch period affects also the fetching of the following opcode as explained above, thus leading to an even more complex fault. Such faults can probably be exploited only if not too many cryptographic transformations or non-critical instructions follow. Otherwise, the not well understood fault becomes too complicated to exploit. For multi-cycle instructions however, the glitch does not necessarily affect the fetching of the new opcode. We obtained the best and most stable results for multi-cycle instructions with memory access, e.g. LD and LPM. Depending on the glitch period, we can rather accurately prevent a given number of bits on the data bus from flipping, although the by far easiest option is to prevent any transition, i.e. to prevent the data bus from updating at all. The effect of such a glitch is that the instruction loads the last value that has been transferred to/from memory.

A typical and straightforward application of the fault results obtained in memory access instructions, again using AES encryption as an example, would be to glitch S-box lookups (typically implemented with LD and LPM commands) to apply some of the well known DFA, CFA or IFA fault analysis techniques in the literature.

VIII. CONCLUSION

We have performed a detailed study of the effects of clock glitches on legacy 8-bit AVR MCUs. The existence of a two-stage pipeline, together with the lack of knowledge about the chip's internal working, limits the level of detail of the characterization of the observed faults. Yet, we have identified faults with a stable behaviour in our experiments. We have shown that instructions can be replaced rather than perfectly skipped, and that the effects of faults are deterministic and reproducible. The easiest targets, both for glitching the fetching and the execution stage, are multi-cycle instructions as it is possible to inject faults that do not affect the full pipeline at the same time. We have shown that such faults can be combined with known attacks, and we have illustrated a straightforward attack that is essentially based on injecting one or a sequence of faults in a single execution.

ACKNOWLEDGMENT

The authors thank Óscar Repáraz for his initial work on the experimental setup. This work was supported in part by the European Commission's ECRYPT II NoE (ICT-2007-216676), by the Belgian State's IAP program P6/26 BCRYPT, by the K.U. Leuven-BOF (OT/06/40) and by the Research Council K.U. Leuven: GOA TENSE (GOA/11/007). Josep Balasch is funded by a PhD grant within the covenant between the universities K.U. Leuven and R.U. Nijmegen. Benedikt Gierlichs is a Postdoctoral Fellow of the Fund for Scientific Research - Flanders (FWO).

REFERENCES

- [1] P. Kocher, "Timing attacks on implementations of Diffie-Hellman, RSA, DSS and other systems," in *Proceedings of CRYPTO*, ser. LNCS, N. Koblitz, Ed., vol. 1109. Springer-Verlag, 1996, pp. 104–113.
- [2] P. Kocher, J. Jaffe, and B. Jun, "Differential power analysis," in *Proceedings of CRYPTO*, ser. LNCS, M. Wiener, Ed., vol. 1666. Springer-Verlag, 1999, pp. 388–397.
- [3] K. Gandolfi, C. Moutrel, and F. Olivier, "Electromagnetic analysis: Concrete results," in *Proceedings of CHES 2001*, ser. LNCS, Ç. Koç, D. Naccache, and C. Paar, Eds., vol. 2162. Springer-Verlag, 2001, pp. 251–261.
- [4] J.-J. Quisquater and D. Samyde, "Electromagnetic analysis (EMA): Measures and counter-measures for smart cards," in *Smart Card Programming and Security (E-smart 2001)*, ser. LNCS, I. Attali and T. P. Jensen, Eds., vol. 2140. Springer-Verlag, 2001, pp. 200–210.
- [5] C. Aumüller, P. Bier, W. Fischer, P. Hofreiter, and J.-P. Seifert, "Fault Attacks on RSA with CRT: Concrete Results and Practical Countermeasures," in *Proceedings of CHES 2002*, B. Kaliski, Ç. Koç, and C. Paar, Eds., vol. 2523. Springer-Verlag, 2002, pp. 260–275.
- [6] S. P. Skorobogatov and R. J. Anderson, "Optical fault induction attacks," in *Proceedings of CHES 2002*, B. Kaliski, Ç. Koç, and C. Paar, Eds., vol. 2523. Springer-Verlag, 2002, pp. 2–12.
- [7] D. Boneh, R. A. DeMillo, and R. J. Lipton, "On the importance of checking cryptographic protocols for faults," in *Proceedings of the 16th annual international conference on Theory and application of cryptographic techniques*, ser. EUROCRYPT'97. Springer-Verlag, 1997, pp. 37–51.
- [8] E. Biham and A. Shamir, "Differential Fault Analysis of Secret Key Cryptosystems," in *Proceedings of the 17th Annual International Cryptology Conference on Advances in Cryptology*. Springer-Verlag, 1997, pp. 513–525.
- [9] L. Hemme, "A Differential Fault Attack Against Early Rounds of (Triple-)DES," in *Proceedings of CHES 2004*, ser. LNCS, M. Joye and J.-J. Quisquater, Eds., vol. 3156. Springer-Verlag, 2004, pp. 170–217.

- [10] J. Blömer and J.-P. Seifert, "Fault Based Cryptanalysis of the Advanced Encryption Standard (AES)," in *Proceedings of Financial Cryptography 2003*, R. Wright, Ed., vol. 2742. Springer-Verlag, 2003, pp. 170–217.
- [11] C. Clavier, "Secret external encodings do not prevent transient fault analysis," in *Proceedings of CHES 2007*, ser. LNCS, P. Paillier and I. Verbauwhede, Eds., vol. 4727. Springer-Verlag, 2007, pp. 181–194.
- [12] H. Bar-El, H. Choukri, D. Naccache, M. Tunstall, and C. Whelan, "The Sorcerer's Apprentice Guide to Fault Attacks," *Proceedings of the IEEE*, vol. 94, no. 2, pp. 370–382, 2006.
- [13] N. Selmane, S. Guilley, and J.-L. Danger, "Practical setup time violation attacks on AES," in *Proceedings of the 7th European Dependable Computing Conference*. IEEE, 2008, pp. 91–96.
- [14] T. Fukunaga and J. Takahashi, "Practical fault attack on a cryptographic LSI with ISO/IEC 18033-3 block ciphers," in *Proceedings of FDTC 2009*. IEEE, 2009, pp. 84–92.
- [15] M. Agoyan, J.-M. Dutertre, D. Naccache, B. Robisson, and A. Tria, "When clocks fail: On critical paths and clock faults," in *Smart Card Research and Advanced Application – CARDIS 2010*, ser. LNCS, D. Gollmann, J.-L. Lanet, and J. Iguchi-Cartigny, Eds., vol. 6035. Springer-Verlag, 2010, pp. 182–193.
- [16] O. Kömmerling and M. G. Kuhn, "Design principles for tamper-resistant smartcard processors," in *Proceedings of the USENIX Workshop on Smartcard Technology*. USENIX, 1999, pp. 9–20.
- [17] R. Anderson and M. Kuhn, "Tamper resistance: a cautionary note," in *Proceedings of the 2nd USENIX Workshop on Electronic Commerce*. USENIX, 1996, pp. 1–11.
- [18] —, "Low cost attacks on tamper resistant devices," in *Proceedings of the 5th International Workshop on Security Protocols*. Springer-Verlag, 1998, pp. 125–136.
- [19] H. Choukri and M. Tunstall, "Round reduction using faults," FDTC workshop 2005, pp. 13–24, 2005.
- [20] C. Kim and J.-J. Quisquater, "Fault attacks for CRT based RSA: New attacks, new results, and new countermeasures," in *Proceedings of WISTP 2007*, ser. LNCS, D. Sauveron, K. Markantonakis, A. Bilas, and J.-J. Quisquater, Eds. Springer-Verlag, 2007, vol. 4462, pp. 215–228.
- [21] J.-M. Schmidt and C. Herbst, "A practical fault attack on square and multiply," in *Proceedings of FDTC 2008*. IEEE, 2008, pp. 53–58.
- [22] A. Barenghi, G. Bertoni, E. Parrinello, and G. Pelosi, "Low voltage fault attacks on the RSA cryptosystem," in *Proceedings of FDTC 2009*. IEEE, 2009, pp. 23–31.
- [23] "ISO/IEC 7816-3: Identification cards - integrated circuit(s) cards with contacts - part 3: Electronic signals and transmission protocols (1997)."
- [24] Xilinx, "XUP Virtex-II Pro Development System User Manual," <http://www.xilinx.com/univ/XUPV2P/Documentation/ug069.pdf>.
- [25] "ISO/IEC 7816-4: Identification cards - integrated circuit(s) cards with contacts - part 4: Organization, security and commands for interchange (2005)."
- [26] T. S. Sho Endo, Naofumi Homma and T. Aoki, "An on-chip glitchy-clock generator and its application to safe-error attack," COSADE workshop, 2011.
- [27] Atmel, "Datasheet ATmega163(L) Complete," <http://www.atmel.com>.
- [28] K. Lemke, K. Schramm, and C. Paar, "DPA on n-Bit Sized Boolean and Arithmetic Operations and Its Application to IDEA, RC6, and the HMAC-Construction," in *Proceedings of CHES 2004*, ser. LNCS, M. Joye and J.-J. Quisquater, Eds., vol. 3156. Springer-Verlag, 2004, pp. 205–219.
- [29] Atmel, "Secure Microcontrollers for Smart Cards. AT90SC Summary," <http://www.atmel.com>.
- [30] NIST, *Advanced Encryption Standard (AES) (FIPS PUB 197)*, National Institute of Standards and Technology, Nov. 2001.